

model that we can reuse for prompt tuning across any number of downstream tasks.

Through LM adaptation, we hope to “quickly” transform T5 into a model more similar to GPT-3, which always outputs realistic text, and is known to respond well to prompts as a “few-shot learner”. It is not obvious how successful this late-stage transformation will be compared to pre-training from scratch, and it has not been investigated previously to our knowledge. As such, we experiment with various lengths of adaptation up to 100K steps.

3 Results

Our frozen models are built on top of pre-trained T5 checkpoints of all sizes (Small, Base, Large, XL, XXL). We leverage the public T5.1.1 checkpoints, which include improvements over the original T5.¹

Our “default” configuration, plotted with a green ‘×’ (—×—) throughout, uses an LM-adapted version of T5 trained for an additional 100K steps, initializes using class labels (see Section 3.2), and uses a prompt length of 100 tokens. While this is longer than the default 10-token prefix used by Li and Liang (2021), our method still uses fewer task-specific parameters, as we only tune the input layer, as opposed to overwriting activations in all network layers. See Figure 4 for a detailed comparison. We will also see shortly that even much shorter prompts are viable as model size increases.

We measure performance on the SuperGLUE benchmark (Wang et al., 2019a), a collection of eight challenging English language understanding tasks.² We report metrics on the development set associated with each dataset.

Each of our prompts train on a single SuperGLUE task; there was no multi-task setup or mixing of training data across tasks. We translate each SuperGLUE dataset into a text-to-text format following Raffel et al. (2020), except that we omit the task names prepended to inputs indicating which SuperGLUE task an example belongs to.

We train our prompts for 30,000 steps using T5’s standard cross-entropy loss, with a constant learn-

ing rate of 0.3 and a batch size of 32. Checkpoints are selected via early stopping on the development set, where the stopping metric is the default metric for the dataset, or the average of metrics for datasets evaluated with multiple metrics. All experiments were run in JAX (Bradbury et al., 2018) using the Adafactor optimizer (Shazeer and Stern, 2018) with weight decay $1e-5$, β_2 decay 0.8, and parameter scaling off. The models were implemented in Flax (Heek et al., 2020). More details are available in Appendix A.

3.1 Closing the Gap

To compare our method with standard model tuning, we tune the public T5.1.1 checkpoints on SuperGLUE using the default hyperparameters specified in the T5 library (learning rate 0.001, and Adafactor optimizer with pre-training parameter states restored). We consider two baselines. (1) “Model Tuning”: For an apples-to-apples comparison, we tune on each task separately, as in our prompt tuning setup.³ (2) “Model Tuning (Multi-task)”: We use T5’s multi-task tuning setup to achieve a more competitive baseline.⁴ In this case, a single model is tuned on all tasks jointly, with a text prefix indicating the task name.

In Figure 1 (p. 1), we see that prompt tuning becomes more competitive with model tuning as scale increases. At the XXL size (11 billion parameters), prompt tuning matches even the stronger multi-task model tuning baseline, despite having over 20,000 times fewer task-specific parameters.

To compare with prompt design, we include GPT-3 few-shot performance on the SuperGLUE dev split, as reported by Brown et al. (2020).⁵ Figure 1 shows that prompt tuning beats GPT-3 prompt design by a large margin, with prompt-tuned T5-Small matching GPT-3 XL (over 16 times larger), and prompt-tuned T5-Large beating GPT-3 175B (over 220 times larger).

³To improve this baseline, we performed a sweep over the batch size hyperparameter and selected 2^{16} tokens per batch.

⁴The T5 SuperGLUE submission used a more complex setup, first mixing multi-task supervised data into pre-training, and then performing single-task fine-tuning. Since we use T5.1.1 throughout, this setup is unavailable, as the pre-training phase is fully self-supervised. We follow Raffel et al. (2020) in using 2^{20} tokens per batch and including DPR data in the multi-task mixture, which is known to boost WSC task performance (Kocijan et al., 2019).

⁵We also experimented with using GPT-3’s manual text prompts directly with our LM-adapted T5 checkpoints. However performance was far below GPT-3 for comparable model sizes. This may be due to differences in pre-training data and model architecture, as well as T5’s shorter sequence length.

¹These improvements are (1) the removal of all supervised data from pre-training, (2) adjustments to hyperparameters d_{model} and d_{ff} , and (3) the use of GeGLU (Shazeer, 2020) over ReLU (Nair and Hinton, 2010) activations.

²The tasks are BoolQ (Clark et al., 2019), CB (De Marneff et al., 2019), COPA (Roemmele et al., 2011), MultiRC (Khashabi et al., 2018), ReCoRD (Zhang et al., 2018), RTE (Dagan et al., 2005; Bar-Haim et al., 2006; Giampiccolo et al., 2007; Bentivogli et al., 2009), WiC (Pilehvar and Camacho-Collados, 2018), and WSC (Levesque et al., 2012).

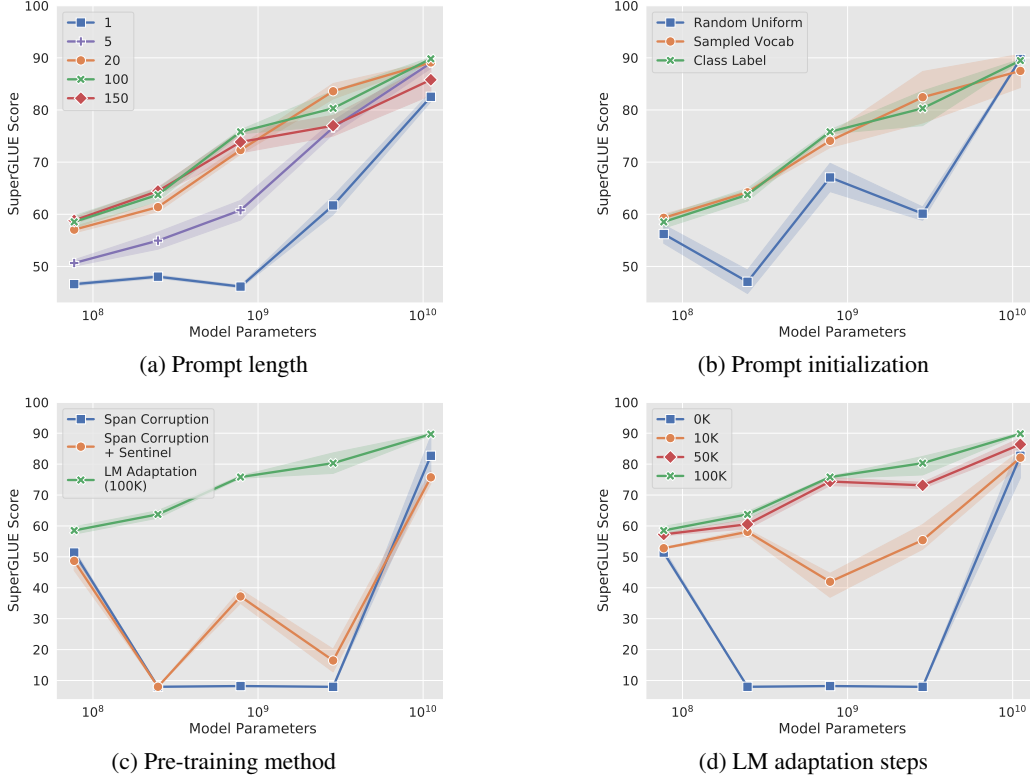


Figure 3: Ablations of various hyperparameters on prompt tuning performance (mean and stddev across 3 runs). In our “default” (—x—) configuration, quality improves stably with model size. Across all ablations, *the largest (XXL) model is the most robust to hyperparameter choice*. (a) **Prompt length**: Increasing to 20+ tokens generally confers a large boost, but XXL performs well even with single-token prompts. (b) **Prompt initialization**: Random uniform initialization lags behind more “advanced” initializations using sampled vocabulary or class label embeddings, but the difference vanishes at XXL size. (c) **Pre-training objective**: LM adaptation outperforms span corruption, even when a sentinel is added to downstream task targets, but XXL works well with any method. (d) **LM adaptation**: Longer adaptation generally gives larger gains, but XXL is robust to even short adaptation.

3.2 Ablation Study

Prompt Length We train prompts for each model size while varying the prompt length in $\{1, 5, 20, 100, 150\}$ and fixing other settings to our **default configuration**. Figure 3(a) shows that for most model sizes, increasing prompt length beyond a single token is critical to achieve good performance. Notably, the XXL model still gives strong results with a single-token prompt, suggesting that the larger the model, the less conditioning signal is needed to achieve a target behavior. Across all models, increasing beyond 20 tokens only yields marginal gains.⁶

Prompt Initialization We ablate the effect of prompt initialization by training models at all sizes while fixing other hyperparameters to their **default values**. For random initialization, we sample uni-

formly from the range $[-0.5, 0.5]$. When initializing from sampled vocabulary, we restrict to the 5,000 most “common” tokens in T5’s SentencePiece vocabulary (Kudo and Richardson, 2018), which is ordered by likelihood in the pre-training corpus. For “class label” initialization, we take the embeddings for the string representations of each class in the downstream task and use them to initialize one of the tokens in the prompt. When a class label is multi-token, we average the token embeddings. At longer prompt lengths, we often run out of class labels before we have initialized all of the prompt tokens. In this case we fall back to our sampled vocab strategy to fill in the prompt.⁷

Figure 3(b) shows our ablation of initialization strategy across model sizes, where we find that

⁶Going past 100 tokens appears mildly detrimental for larger models. A similar pattern of diminishing performance past a certain prefix length is observed by Li and Liang (2021).

⁷T5’s handling of the ReCoRD and WSC tasks requires the model to generate short, free-form text. In these cases, we initialize the prompts with words related to the task: *common-sense*, *reasoning*, *reading*, and *comprehension* for ReCoRD and *commonsense*, *pronoun*, and *resolution* for WSC.

the class based initialization performs best. At smaller model sizes, there are large gaps between the different initializations, but once the model is scaled to XXL size, those differences disappear.

With “class label” initialization, we observe that the class labels typically persist in the learned prompts, such that the nearest token embeddings (in cosine distance) match the tokens used for initialization. Beyond this, we did not find our learned prompts to be interpretable, similar to those of Shin et al. (2020). See Section 7 for details.

Pre-training Objective In Figures 3(c) and 3(d), we see pre-training objective has a clear effect on prompt tuning quality. As hypothesized in Section 2.2, T5’s default “span corruption” objective is not well-suited for training frozen models to be later conditioned by prompts. Intuitively, models pre-trained to read and write sentinel tokens are hard to apply directly to tasks of reading and writing text without sentinels. As seen in Figure 3(c), even the “workaround” of adding a sentinel to the downstream targets has little benefit. While LM adaptation adds value across all model sizes, we note our largest XXL model is the most forgiving and gives strong results even with span corruption.

Given the benefit of LM adaptation, we also explore how long of an adaptation is helpful. Figure 3(d) shows that longer adaptation provides additional gains, up to 100K steps. This suggests that the “transition” from span corruption to a language modeling objective is not a trivial change, and making an effective switch takes an investment of training resources (10% of the steps of the original T5 pre-training). At the same time, as in our other ablations, we observe that the XXL model is robust to even non-ideal configurations. At this size, the gains from adaptation are quite modest.

In the non-optimal “span corruption” setting, we observe instability across model sizes, with the Small model outperforming the larger Base, Large, and XL models. On inspection, we find that for many tasks, these mid-sized models never learn to output a legal class label and thus score 0%. The two most common error modes are copying sub-spans from the input and predicting an empty string. Furthermore, this poor performance is not due to random variance in prompt tuning, as we observe low variance across 3 runs for each size. These results indicate that using models pre-trained with the “span corruption” objective can be unreliable, with only 2 out of 5 models working well, whereas

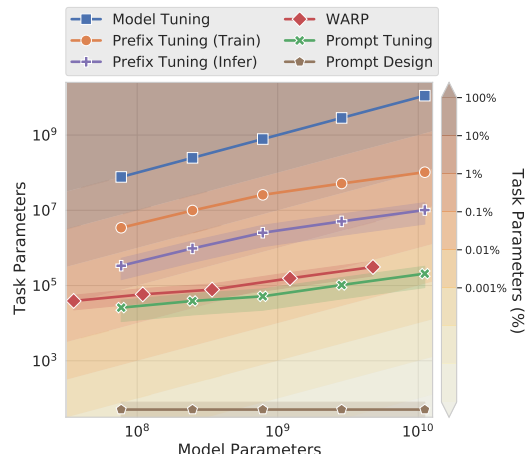


Figure 4: Parameter usage of various adaptation techniques, fixing architecture to T5.1.1 and prompt/prefix length to 1–100 tokens (bands show mean and stddev).

Model Tuning: All parameters are task-specific. **Prefix Tuning:** Activations are tuned in the prefix of each layer, requiring 0.1–1% task-specific parameters for inference, but more are used for training. **WARP:** Task parameters are reduced to under 0.1% by only tuning input and output layers. **Prompt Tuning:** Only prompt embeddings are tuned, reaching under 0.01% for most model sizes. **Prompt Design:** Only a sequence of prompt IDs (500–2000 tokens) is required.

the LM adapted versions work reliably across all model sizes.

We have released T5 1.1 checkpoints adapted using the LM objective for 100K steps for all model sizes.⁸

4 Comparison to Similar Approaches

In this section, we review recent work on learning continuous prompts, and draw comparisons with our method. One important axis of comparison is the number of task-specific parameters each method requires, as shown in Figure 4. Among methods with learnable parameters, prompt tuning is the most parameter efficient, requiring less than 0.01% task-specific parameters for models over a billion parameters.⁹

Li and Liang (2021) propose “prefix tuning”: learning a sequence of prefixes that are prepended

⁸https://github.com/google-research/text-to-text-transfer-transformer/blob/main/released_checkpoints.md#lm-adapted-t511lm100k

⁹To compare with prompt design, we count each token in the prompt as a parameter, and assume a prompt of between 500–2000 tokens to match the GPT-3 setting. While this technique is by far the most parameter efficient, it comes at the cost of task quality.